Coding-Basic-HW-2

Student ID: 113062362

Date: October 10, 2026

---

ozdgemm: Adaptive FP64 GEMM Emulation and HPL

NCHC nano4, one NVIDIA H200, CUDA 13.1, cuBLAS 13.2, NVHPC 26.3

This project implements FP64 matrix multiplication using INT8 Tensor Cores, chooses precision and backend per call, and integrates the library into netlib HPL. On one H200, it reaches 1.51 times native DGEMM speed for large square matrices. HPL gains little because host work and transfers dominate. The report explains the implementation, tests, profiling, and limits.

1. Problem and environment

The task is to compute $C = \alpha AB + \beta C_0$ with FP64-level accuracy using low-precision Tensor Cores. Here α and β are scalar coefficients, and C_0 is the input matrix. The main questions are how many integer products are needed, how to select them safely, and when their conversion and reconstruction costs are worthwhile. The deliverables include a reusable library, correctness tests, native/v0/final comparisons, HPL results, and reproducible source.

All measurements here use one H200 (sm_90, 700 W). Its FP64 Tensor Core peak is 67 TFLOPS and dense INT8 peak is 1,979 TOPS. Measured cuBLAS rates are about 56 TFLOPS and 1,350 TOPS, respectively. Strong native FP64 makes this a demanding test for emulation. RTX 5090 values in the repository are projections, with no measured RTX 5090 results.

The TA confirmed that HPL should come from netlib, that "starter code" was a typo, and that H200 measurements were accepted. v0 is our first DGEMM implementation. The October 7 discussion was completed.

2. Method

2.1 Integer products and reconstruction

Ozaki scheme II [1, 2] uses the Chinese remainder theorem (CRT) to reconstruct a large integer product from small modular products. Scale each row of A and column of B by powers of two, then truncate to integer matrices A' and B'. Power-of-two scaling is exact in the normal range. Truncation and the final FP64 arithmetic determine the accuracy budget.

Choose pairwise-coprime moduli no larger than 256. Residues use a balanced signed representation, so 200 modulo 256 is stored as -56. The range for modulus 256 is [-128, 127], with symmetric ranges for odd moduli. After GEMM, imod reduces each signed INT32 output to [0, m). This reduction is fused into CRT reconstruction.

Let M be the product of the moduli. The retained precision is $P = \left\lfloor \frac{\lfloor \log_2 M \rfloor - 2}{2} \right\rfloor$. Scaling keeps each input row/column norm below 2^P, giving $|A'B'|_{ij} < 2^{2P} \leq \frac{M}{4}$. Thus the integer product lies inside the signed CRT range. Fourteen moduli give P = 54.

Table 1. One ozd_dgemm call.

Stage	Implementation	Purpose
Scale	Choose row and column exponents	Bound the integer product
Convert	Truncate and store balanced INT8 residues	Fit each modular input in INT8
Multiply	One INT8 GEMM per modulus, INT32 accumulation	Compute exact modular products
Reconstruct	Reduce signed outputs and apply CRT	Recover the signed integer product
Unscale	Restore scales, apply α and β	Return the FP64 update

For residues c_t under moduli m_t, reconstruct:

$$X = \left(\sum_{t=1}^{N} c_t S_t \right) \bmod M, \qquad S_t = M_t \left(M_t^{-1} \bmod m_t \right), \qquad M_t = \frac{M}{m_t}$$

The modulus product exceeds 2^{100}, so CRT coefficients are split into 40-bit chunks whose sums fit the exact FP64 integer range. The final signed result uses TwoSum. Integer modulo reduction and CRT are fused to avoid a separate pass over the INT32 outputs. The implementation is in src/ozdgemm.cu, using cuBLAS INT8 GEMM and native DGEMM [4].

2.2 Accuracy and adaptive precision

Let C_0 denote the input C and let u = 2^{-53}. Exact scaling and integer reconstruction give a truncation term proportional to 2^{-P}. The fixed-moduli tests use this full budget:

$$E_{\text{budget},ij} = 4\sqrt{k}\, 2^{-P} |\alpha| \|A_i\|_2 \|B_j\|_2 + 4u \left(|\alpha||A||B| + |\beta C_0| \right)_{ij}$$

The first term covers input truncation and the second allows final FP64 rounding, including βC_0. Adaptive tests replace the first term with $\tau|\alpha|(|A||B|)_{ij}$, keeping the same rounding term. This is a tested budget under finite arithmetic without overflow or significant underflow. The tests do not prove a universal bound for every FP64 input.

ozd_set_accuracy(tol) controls the truncation term. For positive tol, $\tau = \text{tol}$. For tol = 0, $\tau = k\, 2^{-53}$ selects a native-scale target, rather than zero error or a bitwise match to native DGEMM. More moduli cannot remove unavoidable final rounding. For example, $A = B = \alpha = \beta = 1$ and $C_0 = 2^{53}$ have exact output $2^{53} + 1$, which FP64 cannot represent.

Choose the fewest moduli that satisfy the truncation condition:

$$4\sqrt{k}\, 2^{-P} R \le \tau, \qquad R = \max_{i,j} \frac{\|A_i\|_2 \|B_j\|_2}{(|A||B|)_{ij}}$$

Level 1 uses one extra INT8 GEMM on scaled magnitudes rounded down to [0, 127]. Restoring row/column scales gives a lower bound on the absolute product and an upper bound on R. Level 2 runs only if level 1

fails. It retains a second band, $h = \lfloor s \rfloor, \quad l = \lfloor 128(s-h) \rfloor, \quad s \geq h + \frac{l}{128}$, and combines hh, hl, lh and ll using four INT8 GEMMs with 64-bit accumulation.

If the estimator still gives a zero lower bound or cannot certify the target within 16 moduli, it returns native DGEMM and reports reason 4. This means no certificate was obtained. It does not imply that the true product needs more than 16 moduli. Native fallback may also miss a requested tolerance.

2.3 AUTO selection and library use

A fitted cost model predicts native and emulated time from m, n, k and the modulus count. AUTO may choose native before scaling if even the fewest admissible moduli are predicted to lose. Later decisions can incur estimator work. ozd_get_info reports the chosen backend, moduli, predictions and fallback reason.

Fitting job 466503 gives 63/63 correct fixed-moduli choices and 15.9% worst prediction error on its training grid. Applying the same model to repeat job 466571 gives 63/63 and 16.8%. These use the same shapes. Unseen shapes remain unvalidated, and AUTO has no universal speed guarantee.

Other fallback codes are 1 for NaN/Inf, 2 for degenerate inputs or resource limits, and 3 when AUTO selects native. The k limit is 131056 to avoid INT32 accumulator overflow. Use one GPU, one compute stream, serialized calls and valid arguments. Constants are shared, so synchronize before switching streams. Argument checks are incomplete, and concurrent emulated handles and device switching are unsupported.

Create handle h on a compute stream
Set backend(h, AUTO) and accuracy(h, tol), or fixed moduli(h, 14)
Reserve workspace(h, m, n, k) before timing
Call ozd_dgemm(h, opA, opB, m, n, k, alpha, A, lda, B, ldb, beta, C, ldc)
Read ozd_get_info(h) for the path, precision and fallback reason

2.4 HPL integration

Build the checksum-verified netlib HPL 2.3 archive [3] with -DHPL_CALL_CBLAS. The shim in hpl/ozd_shim.cu defines cblas_dgemm. Updates with M, N at least 1024 and K at least 128 use the GPU. Smaller updates stay on CPU BLAS.

Netlib HPL stores its matrix on the host, so GPU updates copy A, B and C over PCIe. A three-line patch page-locks the matrix and pre-allocates device buffers before the timed solve (late_allocs = 0). The shim processes C in 2048-column chunks on three streams, overlapping transfers with serialized GEMM. Fixed-moduli results match the serial path bitwise.

3. Data and correctness

3.1 Inputs and measurement controls

Uniform inputs in [-0.5, 0.5) model HPL-like data. Wide-exponent inputs use $(\text{rand} - 0.5)\exp(\varphi\,\text{randn})$ with φ = 0.5, 1, 2 or 4. Adversarial cases include rows scaled by $2^{\pm 300}$, zero vectors, NaN/Inf, odd shapes, padded leading dimensions and all four transpose combinations. Benchmarks cover squares of size 1024-16384, HPL update shapes with k = 256-2048, and skinny/tall matrices.

Each Slurm job records its host, GPU, clocks, driver and experimental commit in results/<job>/env.txt. Native, v0 and final are compared in the same job on the same GPU. CUDA events time the whole library call, using the median of 10 calls after three warm-ups. Inputs stay on the GPU and workspace is reserved before timing. Nsight stage timings come from separate profiled calls.

Native is re-timed after each shape and slowed by up to 7%, making the reported speedups conservative. A/B runs alternate builds with matching warm-ups. HPL totals vary by up to about 20% across shared-node jobs, so configuration comparisons use one job and the shim's offloaded wall time.

3.2 Correctness results

The reference in tests/common.cuh accumulates in double-double arithmetic and stores the result as FP64. Every emulated entry is checked against the fixed or adaptive budget, including final rounding. This FP64-rounded reference provides evidence on the tested data, rather than an exact real-arithmetic oracle.

Job 466571 reports 36 matrix cases, four dispatcher checks, blocking invariance and determinism, with **zero failures** and worst error/budget ratio 0.32. NaN/Inf fallback preserves IEEE placement. Repeated calls and column blocking give bitwise-identical results.

The adaptive sweep has 108 cases (jobs 461134, 462933, 466503), with k = 256, 1024 or 4096 and tol from 1e-15 to 1e-3. All 79 emulated cases pass, with largest stored error/budget ratio **0.057**. Uniform data selects 14 moduli at tol = 0, 11 at 1e-9 and 8 at 1e-6.

Table 2. Selected checks, job 466571.

Case	moduli	emu error	native error	error/budget	path
Uniform, 512 cubed	14	3.8e-16	4.7e-16	0.052	emulated
φ = 2, tol = 1e-6	11	9.8e-10	5.7e-15	0.054	level 2
φ = 4, tol = 0	-	4.3e-15	4.3e-15	-	native

The error columns are normalized by $|\alpha|(|A||B|)_{ij} + |\beta(C_0)_{ij}|$, taking the largest entrywise ratio. Error/budget includes FP64 rounding.

4. Performance and profiling

4.1 GEMM compared with native and v0

Speedup is native median time divided by emulated median time. Values above one favour emulation. Job 466571 compares the initial v0 and final implementations under the same controls. Large square matrices amortize conversion and CRT, while HPL's small inner dimension limits the benefit.

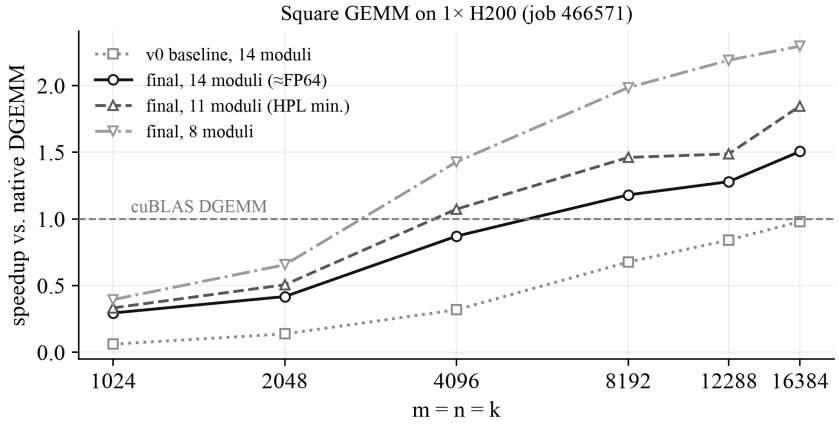


Fig. 1. Square GEMM, job 466571. Fourteen moduli win from about 8192 cubed. Eleven moduli win earlier with lower accuracy.

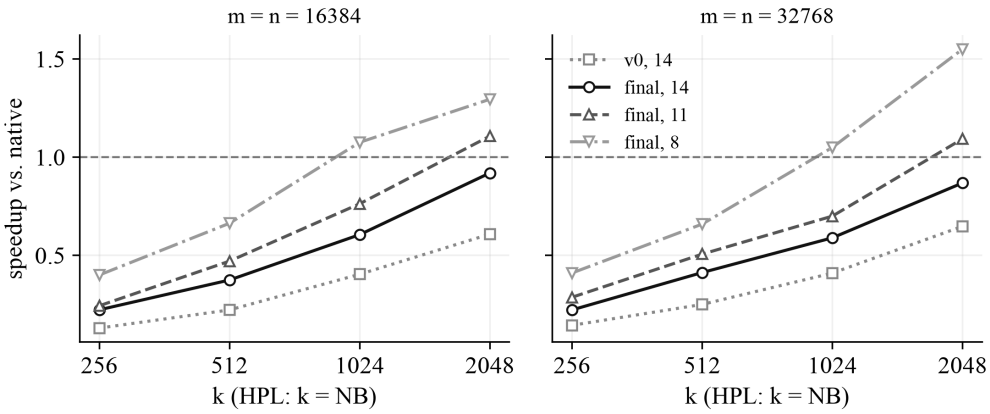


Fig. 2. HPL updates, job 466571. With 14 moduli, emulation loses at every tested k up to 2048.

Table 3. Same-job GEMM comparisons, job 466571.

Shape m x n x k	native ms	v0, 14	final, 14	final, 11	final, 8
16384 cubed	163.4	0.98x	**1.51x**	1.85x	2.30x
8192 cubed	19.5	0.68x	**1.18x**	1.46x	1.98x
32768 x 32768 x 1024	39.7	0.41x	0.59x	0.70x	1.05x
16384 x 64 x 16384	0.68	0.02x	0.08x	0.10x	0.13x

With 14 moduli, the measured INT8/FP64 throughput ratio of about 24 suggests an arithmetic-only limit near 24/14 = 1.7 times native speed. The measured 1.51 is about 88% of this estimate. The model omits conversion and reconstruction.

Profiling. At 32768 x 32768 x 1024, Nsight job 466571 records INT8 GEMM 35.5 ms (54%), CRT 28.5 ms (43%), conversion 1.4 ms and scaling 0.25 ms, against native 38.3 ms in that profiled call. Fourteen INT32 outputs total about 60 GB. CRT reads them at about 2.4 TB/s. At small k, this memory traffic dominates. At large k, useful INT8 arithmetic dominates.

Optimizations. Conversion combines scaling with residue generation. CRT fuses modulo reduction, reconstruction and the final update. For k at least 2048 and m, n at least 1024, separate GEMMs per modulus let cuBLAS choose a 128 x 256 tile. This is 1.1-1.2 times faster than strided-batched GEMM in A/B job 466399. Smaller calls keep batching to avoid extra launches. A proposed CUTLASS modulo epilogue was dropped because its estimated effect on total HPL speed was below 1%.

4.2 HPL accuracy

HPL passes when its scaled residual is below 16. Job 461194 uses N = 20000 and NB = 1024. Eleven moduli is the smallest tested count that passes, with residual 9.6. Ten moduli fails at 165. This threshold is specific to the tested configuration.

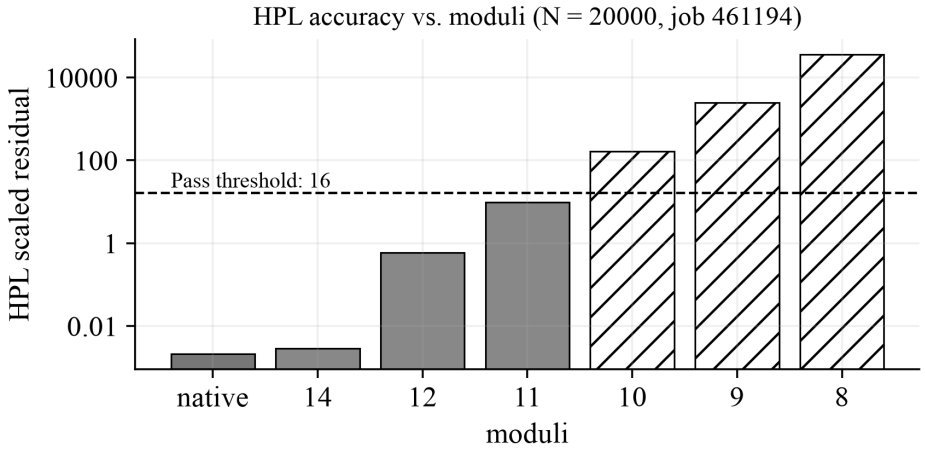


Fig. 3. HPL residuals, job 461194. Removing a modulus raises residuals roughly 16 times. Hatched bars fail the threshold of 16.

4.3 Netlib HPL speed

Serial-offload job 461201 uses N = 60000 and NB = 1024. GPU GEMM is only 2.5 s of the native 66.5 s total. Most time is spent on transfers, CPU BLAS and other HPL work. The serial components can be stacked without counting overlapping work twice.

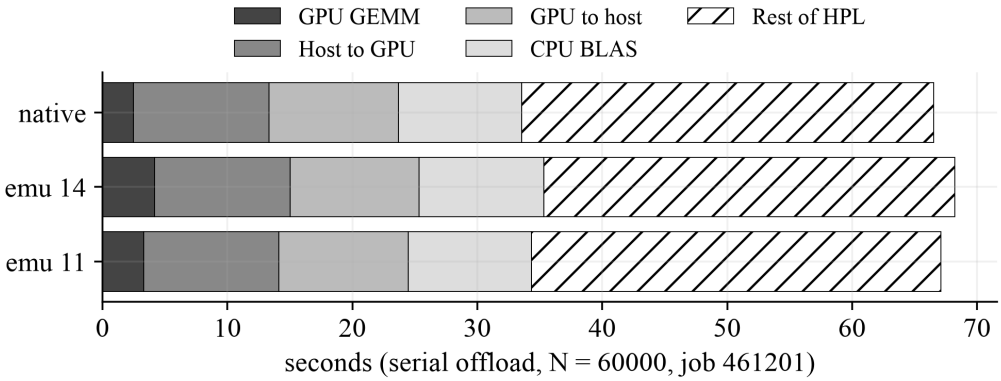


Fig. 4. Serial HPL, job 461201. GPU GEMM is 3.7% of total time, so faster arithmetic has little effect on the score.

Table 4. Serial HPL, N = 60000, job 461201.

Backend	time s	GFLOPS	vs native	residual
native	66.51	2,165	1.000	2.3e-3
emulation, 14 moduli	68.21	2,111	0.975	3.1e-3
emulation, 11 moduli	67.12	2,146	0.991	6.3
AUTO, tol = 0	66.47	2,167	1.001	2.3e-3

Overlapping transfers and GEMM cuts native HPL time by 14% and raises GFLOPS by 16.6% (jobs 461491, 462597). Emulated work then hides behind transfers, with offloaded wall times of 8.15 s versus 8.04 s native.

Fewer moduli reduce GEMM time without reducing total wall time (job 463015). AUTO at tol = 0 selects native on all 57 calls in the serial test.

Profiling also found a NaN/Inf flag read-back that serialized overlap. Writing the flag to mapped pinned memory reduced emulated GEMM time from 5.30 s to 2.79 s. This improved the integration without changing the host-memory bottleneck.

4.4 NVIDIA HPL comparison

Job 514537 runs NVIDIA HPC-Benchmarks 26.02.01 [5] with Apptainer, one rank, one H200 and the same 12 CPU cores/node as our netlib runs. NB is 1024. NVIDIA rounds the requested N = 50000 down to 49152, so these are similar problem sizes rather than identical matrices.

Table 5. Same-node H200 comparison, job 514537.

Implementation	N	time s	TFLOPS	residual
NVIDIA HPL, native	49,152	2.02	**39.25**	3.5e-4
NVIDIA HPL, larger N	131,072	30.30	49.55	7.6e-4
Netlib + shim, native	50,000	41.10	2.03	2.1e-3
Netlib + shim, 14 moduli	50,000	40.81	2.04	2.4e-3

NVIDIA HPL is about 19 times faster near N = 50000. Its standalone DGEMM rate is 52.8 TFLOPS, close to our cuBLAS rate. The large HPL gap comes from keeping the matrix and panel work on the GPU. Netlib transfers host-resident updates, with DGEMM only about 4% of total time. Even eliminating that work would improve its score by only about 4%.

Requesting NVIDIA FP64 emulation on H200 produces a warning that it requires SM100 or newer, then runs native. The recorded residual is bitwise identical to native, and 39.45 versus 39.25 TFLOPS is not evidence of emulation speedup. At N = 131072, NVIDIA HPL reaches about 94% of its DGEMM rate. A faster GEMM can matter in that setting, but our 14-moduli path loses on the tested small-k H200 updates.

5. Limits and applicable scope

On H200, emulation helps large squares from about 8192 cubed at 14 moduli, or smaller cases at looser accuracy targets. Small k, skinny matrices and small squares are slower than native because launch and INT32-output costs dominate. A GPU with weaker FP64 could offer more benefit, but RTX 5090 projections require hardware validation. HPL also needs GPU-resident data and a substantial GEMM share to benefit.

For the checked $\varphi = 2$ sample at tol = 0, available retained precision is insufficient. For $\varphi = 4$, level 2 still produces a zero lower bound at every tested tolerance. Its stored log2_ratio = 1048576 is an unusable-bound sentinel, not the true ratio. Tight targets such as 1e-15 can also exceed native accuracy. All fallbacks are reported, rather than treated as certified emulated results.

6. AI collaboration

The student set goals, chose designs and reviewed results. Claude Code (Anthropic) wrote most code and scripts, ran Slurm jobs through a relay CLI, and drafted reports from saved logs. Codex revised the report and plots. External code is netlib HPL and cuBLAS, with NVIDIA's container used for comparison. GEMMul8 and cuBLAS FP64 emulation are not used in the library.

Workflow: define the reference and per-entry checks, ask the agent for a small change, review the source, run controlled jobs, inspect logs/profiles, then accept or undo it. Each number must cite a saved job, and optimizations must preserve correctness and determinism. Matching warm-ups exposed a false 6-8% regression caused by GPU heating. Bound analysis corrected overstated accuracy claims on wide-exponent data.

7. Reproduction and source records

Repository: https://github.com/xs1128/coding-hw2. Import commit: e93c2e8. Job 466571 records final ef7e39b and v0 295009a. Their original history is missing, so v0 cannot yet be rebuilt. These report/audit revisions remain uncommitted.

```
cd hw2
module load x86-nvhpc/26.3
make -j
sbatch scripts/bench.sbatch
bash scripts/build_hpl.sh
N=50000 CONFIGS="native emu:14 emu:11 autotol:0" sbatch scripts/hpl.sbatch
apptainer pull hpc-benchmarks-26.02.01.sif docker://nvcr.io/nvidia/hpc-benchmarks:26.02.01
SIF=$PWD/hpc-benchmarks-26.02.01.sif sbatch scripts/nvhpl.sbatch
```

Source layout: include/ API, src/ library, tests/ checks, hpl/ shim/patch, scripts/ jobs and analysis. CSVs, logs and profiles are under results/<job>/. Figure and table captions identify their source jobs.

Job 514579 is excluded because its hardware label conflicts with the confirmed project setup. Its original files remain unchanged. Detailed arithmetic checks, proof limits and source checks are in docs/submission-audit.md. Local CPU/log/source checks passed, but CUDA was not rerun. A video script is included. The actual recording is still required and must be under 10 minutes.

8. References

[1] Ozaki, Uchino and Imamura. Ozaki Scheme II, 2025. https://arxiv.org/abs/2504.08009.

[2] Uchino, Ozaki and Imamura. Performance enhancement of the Ozaki scheme on integer matrix multiplication unit, 2025. https://doi.org/10.1177/10943420241313064.

[3] Petitet et al. HPL 2.3, netlib, 2018. https://www.netlib.org/benchmark/hpl/.

[4] NVIDIA. cuBLAS documentation. https://docs.nvidia.com/cuda/cublas/.

[5] NVIDIA. NVIDIA HPC Benchmarks, HPL documentation. https://docs.nvidia.com/nvidia-hpc-benchmarks/HPL_benchmark.html.